

Université Ibn Zohr
École Supérieure de Technologie – Agadir
Filière : Conception et Développement Logiciel
Année universitaire : 2025-2026

Rapport de SFE

Conception et développement d'une plateforme interne de gestion
et d'encadrement des stagiaires
Application : StageFlow

Réalisé par : Reda Bouzid
Date de soutenance : 28/05/2026
Encadrant : Neuman Charhbili

Table des matières

1	Introduction générale	4
2	Présentation du projet	5
2.1	Contexte général et justification	5
2.2	Objectifs du projet	5
2.3	Périmètre fonctionnel	6
3	Analyse fonctionnelle détaillée	7
3.1	Identification des acteurs et responsabilités	7
3.2	Besoins fonctionnels	7
3.2.1	Gestion de l'authentification et des comptes	7
3.2.2	Gestion des stages et des comptes étudiants	8
3.2.3	Gestion de l'encadrement et du suivi des tâches	8
3.2.4	Gestion des rapports de stage	8
3.3	Diagrammes UML	8
3.3.1	Diagramme des cas d'utilisation	8
4	Parcours utilisateurs critiques	10
4.1	Parcours étudiant	10
4.2	Parcours superviseur	10
5	Architecture technique	13
5.1	Architecture globale	13
5.2	Stack technologique	14
5.3	Sécurité et validation	14
6	Interfaces utilisateur et captures d'écran	15
6.1	7.1 Authentification — Écran de connexion	15
6.2	7.10 Sécurité, fiabilité et qualité	16
6.2.1	Mécanismes de sécurité	16
6.2.2	Qualité et robustesse	16
6.3	7.2 Tableau de bord administrateur	16
6.4	7.3 Gestion des stagiaires	17
6.5	7.4 Création de stage	17
6.6	7.5 Gestion des tâches	17
6.7	7.6 Suivi de progression	18
6.8	7.7 Espace étudiant	18
6.9	7.8 Gestion des rapports	19
6.10	7.9 Workflow d'encadrement (vue synthétique)	19

7	Outils et technologies utilisés	20
7.1	8.1 Git et GitHub	20
7.2	8.2 Node.js	21
7.3	8.3 Express.js	21
7.4	8.4 PostgreSQL	22
7.5	8.5 React et Vite	23
7.6	8.6 Docker	23
7.7	8.7 JWT (JSON Web Token)	24
7.8	8.8 Joi	24

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant, Neuman Charhbili, pour son accompagnement rigoureux, ses orientations méthodologiques et sa disponibilité constante tout au long de ce projet de fin d'études. Je remercie également l'ensemble du corps enseignant de l'École Supérieure de Technologie – Agadir pour la qualité de la formation dispensée, qui m'a permis d'acquérir des bases solides en ingénierie logicielle, en architecture des systèmes et en gestion de projet. Mes remerciements vont aussi à ma famille et à mes proches pour leur soutien moral, leur patience et leur encouragement durant toute la durée de ce travail.

Résumé

StageFlow est une plateforme interne de gestion et de suivi des stagiaires conçue pour structurer et automatiser le processus d'encadrement académique et professionnel au sein d'une institution. La plateforme centralise l'ensemble du cycle de vie du stage : création du stage par le superviseur, génération du compte étudiant, suivi des tâches assignées, encadrement continu par remarques et validation des rapports finaux.

L'application adopte une architecture full-stack moderne avec un backend Node.js/Express sécurisé par JWT et RBAC, une validation stricte des données via Joi, et un frontend React/Vite offrant des interfaces adaptées aux trois rôles : Administrateur, Superviseur et Étudiant. La persistance des données est assurée par PostgreSQL avec un modèle relationnel robuste et auditable.

Le système opère selon un flux clair : après une acceptation externe du stage, le superviseur crée le compte étudiant et envoie automatiquement une invitation d'activation par courriel. L'étudiant accède ensuite à la plateforme pour consulter ses tâches, suivre son avancement et soumettre son rapport. Toutes les interactions sont historisées pour assurer la traçabilité complète et l'auditabilité des décisions.

Mots-clés : gestion interne des stagiaires, plateforme d'encadrement, trois rôles, architecture MVC, JWT, RBAC, React, Node.js, PostgreSQL, supervision académique.

Chapitre 1

Introduction générale

La gestion des stagiaires au sein d’une institution ou d’une structure d’accueil requiert une coordination rigoureuse entre plusieurs acteurs : superviseurs de stage, administrateurs et étudiants. Lorsque ces activités demeurent fragmentées (courriels, feuilles de calcul, dossiers isolés), le suivi devient inefficace, les décisions manquent de traçabilité et l’encadrement pédagogique s’en trouve compromis.

Le projet StageFlow répond à cette problématique en proposant une plateforme interne de gestion des stagiaires, spécifiquement conçue pour centraliser et structurer le processus d’encadrement. Son objectif n’est pas d’automatiser le recrutement ou la sélection des candidats (qui restent des processus externes ou manuels), mais de fournir un cadre fiable pour la gestion opérationnelle des stages une fois les étudiants acceptés : création et pilotage des stages, génération des comptes étudiants, assignation des tâches, suivi continu de la progression, collecte des remarques pédagogiques et validation des rapports.

Cette approche améliore la communication entre superviseurs et étudiants, garantit la traçabilité complète des décisions et actions, et renforce la cohérence de l’encadrement académique. La plateforme s’appuie sur une architecture full-stack moderne avec une authentification sécurisée (JWT), un contrôle d’accès par rôles (RBAC) et une validation stricte des données (Joi).

Ce rapport présente successivement la vision du projet, l’analyse fonctionnelle détaillée, les parcours utilisateurs critiques, l’architecture technique MVC retenue, puis les dispositifs de sécurité, fiabilité et qualité implémentés pour garantir la robustesse et l’auditabilité de l’application.

Chapitre 2

Présentation du projet

2.1 Contexte général et justification

Dans de nombreuses institutions, la gestion des stagiaires souffre d'une absence d'intégration entre les outils utilisés par les superviseurs, les administrateurs et les étudiants. Cette dispersion de l'information crée des ruptures opérationnelles : informations perdues entre courriels, validations tardives, historique incomplet des décisions et difficulté à mesurer l'avancement réel des missions et du travail accompli. Ces limites affectent à la fois l'efficacité opérationnelle de l'institution et la qualité de l'expérience vécue par l'étudiant stagiaire.

La justification de StageFlow repose sur la nécessité d'un système unifié et interne, capable d'orchestrer de manière fiable et auditable les différentes phases du suivi d'un stage : enregistrement du stage, affectation et suivi des tâches, encadrement continu, collecte des remarques pédagogiques et validation finale des rapports. En centralisant les données et en clarifiant les responsabilités par rôle, la plateforme réduit les ambiguïtés, améliore la communication entre superviseurs et étudiants, et renforce la gouvernance et la traçabilité du processus.

2.2 Objectifs du projet

Objectif principal

Concevoir, développer et déployer une plateforme web full-stack interne dédiée à la gestion et au suivi des stagiaires, permettant aux superviseurs de créer et de piloter les stages, de gérer les comptes étudiants, d'assurer le suivi des tâches assignées, de fournir un encadrement continu et de valider les rapports finaux, tout en garantissant une sécurité robuste, une traçabilité complète et une auditable des actions.

Objectifs spécifiques

- Fournir un flux opérationnel clair où le superviseur crée le stage et les comptes étudiants après acceptation externe, et reçoit automatiquement une invitation d'activation.
- Automatiser l'envoi des notifications (invitations, rappels) et conserver un historique complet et auditable de toutes les actions et décisions.
- Mettre en place une sécurité robuste par authentification JWT et contrôle d'accès par rôles (RBAC), et valider strictement les données à l'entrée via Joi.
- Mettre à disposition des superviseurs et de l'administrateur des tableaux de bord et des indicateurs pour le suivi pédagogique, la progression des tâches et l'état des rapports.
- Concevoir une architecture modulaire (API REST, MVC) et testable facilitant la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités.

2.3 Périmètre fonctionnel

Le périmètre fonctionnel de StageFlow couvre les besoins cœur du processus de suivi des stagiaires au sein d'une institution, avec une séparation claire entre les responsabilités de chaque acteur et un chaînage logique des opérations. La plateforme prend en charge :

- L'authentification fiable, l'autorisation basée sur les rôles et la gestion des profils utilisateurs.
- L'administration des entités cœur : étudiants/stagiaires, superviseurs et structures d'accueil (sociétés, organisations).
- La création et la gestion des stages par les superviseurs (description, dates, objectifs, encadrement), ainsi que la gestion manuelle des comptes étudiants associés.
- L'assignation et le suivi du travail via des tâches avec statuts de progression et retours d'encadrement du superviseur.
- La gestion documentaire du rapport de stage : rédaction par l'étudiant, soumission, revue par le superviseur et décision finale (validation ou demande de corrections).
- L'audit et la supervision par des indicateurs pour appuyer le pilotage pédagogique et administratif.

Le système n'inclut pas de module de publication d'offres, de recherche d'opportunités ou de candidature autonome par les étudiants (ces processus relèvent d'une gestion externe ou manuelle antérieure à l'entrée dans StageFlow). Les fonctionnalités avancées hors périmètre cœur (intégrations complexes, analytique prédictive, signatures électroniques) ne constituent pas le principal de cette version, mais l'architecture retenue en facilite l'ajout progressif.

Chapitre 3

Analyse fonctionnelle détaillée

3.1 Identification des acteurs et responsabilités

Le système met en interaction trois acteurs principaux, chacun disposant d'un champ de responsabilité clairement défini afin d'éviter les chevauchements décisionnels et assurer une gouvernance efficace.

L'Administrateur assure la gouvernance technique et administrative de la plateforme : gestion des comptes et des permissions, surveillance de la cohérence et de l'intégrité des données, contrôle des accès, consultation des logs et supervision des incidents de sécurité. Les comptes administrateurs sont créés manuellement au niveau de la base de données par les opérateurs du service, garantissant un contrôle strict de ce rôle privilégié.

Le Superviseur est l'acteur opérationnel central. Il s'inscrit librement via l'authentification standard et dispose de l'ensemble des responsabilités d'encadrement : création et gestion des stages (description, dates, objectifs, affectations), création des comptes étudiants nécessaires, assignation des tâches granulaires, suivi de l'avancement, fourniture de remarques pédagogiques et expertise sur le rapport soumis. Lorsqu'un superviseur crée un compte étudiant, le système envoie automatiquement un courriel d'activation permettant à l'étudiant de définir son mot de passe et accéder à la plateforme.

L'Étudiant (ou stagiaire) dispose d'un compte créé par le superviseur et accède à son espace personnel dédié au stage. Il consulte les tâches assignées, met à jour son avancement, consulte les remarques du superviseur et soumet son rapport final via l'interface. Chaque action est historisée pour assurer la traçabilité complète et permettre aux superviseurs et à l'administrateur d'auditer le suivi pédagogique.

Cette répartition des trois rôles soutient une gouvernance responsable et auditable du processus, où chaque action est traçable et reliée à un acteur identifié.

3.2 Besoins fonctionnels

3.2.1 Gestion de l'authentification et des comptes

Le système doit permettre l'authentification, la gestion de session et la récupération de compte de manière fiable et sécurisée. Dans un environnement multi-rôles, la gestion des identités ne se limite pas à la connexion ; elle implique aussi un contrôle fin des permissions selon les responsabilités métier assignées à chaque rôle.

L'application implémente une authentification par jeton JWT (JSON Web Token), accompagnée de mécanismes de validation stricte des entrées, de vérification de l'état du compte et de protection des routes sensibles. La gestion des mots de passe suit des pratiques de sécurité solides : hachage bcrypt, règles de robustesse, possibilité de renouvellement, afin de réduire les risques de compromission des accès.

3.2.2 Gestion des stages et des comptes étudiants

Dans StageFlow, la création et la gestion des stages sont réalisées par les superviseurs. Un stage comporte une description claire, des dates de début et de fin, des objectifs opérationnels et les éléments d'encadrement (superviseur responsable, lieu, structure d'accueil). Le superviseur crée manuellement les comptes étudiants associés aux stages ; chaque création de compte déclenche automatiquement l'envoi d'un courriel d'activation permettant à l'étudiant de définir son mot de passe et accéder à la plateforme.

Le système conserve un historique complet des modifications (qui a créé/modifié quel stage, dates d'action, commentaires), assurant la traçabilité et l'auditabilité. Le système ne gère pas de module de publication d'offres ni de candidatures autonomes : l'admission au stage et la création du compte étudiant relèvent d'un processus externe antérieur ou d'un traitement manuel par le superviseur basé sur des critères de sélection hors plateforme.

3.2.3 Gestion de l'encadrement et du suivi des tâches

Une fois le compte étudiant créé et le stage enregistré, la plateforme structure le travail opérationnel par des tâches. Le superviseur décompose la mission en tâches granulaires, fixe des échéances et observe l'évolution des statuts (créée, en cours, terminée, retour attendu, etc.). L'étudiant accède à son espace personnel pour mettre à jour l'avancement de ses tâches, signaler des blocages et recevoir des remarques exploitables du superviseur.

Le système maintient un historique complet des interactions : commentaires, changements de statut, dates de mise à jour, auteur de chaque action. Cette traçabilité soutient l'évaluation continue et permet une capitalisation des retours d'expérience en fin de cycle.

3.2.4 Gestion des rapports de stage

La gestion des rapports constitue la phase de clôture du stage. L'étudiant rédige et soumet son document final via l'interface (format texte, fichier ou autre selon la plateforme). Le superviseur dispose d'un workflow de revue lui permettant d'examiner le rapport, de l'accepter ou de demander des corrections et des révisions, assurant la qualité pédagogique et professionnelle des livrables.

Le système garantit l'intégrité de ce cycle en empêchant les validations incohérentes, en enregistrant précisément l'auteur de chaque décision, la date de traitement et les remarques associées. Cette traçabilité est essentielle pour sécuriser l'évaluation académique et professionnelle, et pour constituer une archive fiable et auditable des livrables de stage.

3.3 Diagrammes UML

Cette section rassemble les diagrammes UML utilisés pour documenter la modélisation et les interactions de la plateforme StageFlow. Les diagrammes fournissent une vue fonctionnelle et structurelle du système, en se concentrant sur les trois rôles implémentés (Administrateur, Superviseur, Étudiant) et leurs interactions avec les services exposés par l'application.

3.3.1 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation ci-dessous synthétise les principaux scénarios fonctionnels pris en charge par StageFlow : création et gestion des stages, création des comptes étudiants par le superviseur, suivi des tâches, encadrement et validation des rapports. Il illustre les services métier offerts par la plateforme sans évoquer de fonctions externes de recrutement ou de publication d'offres.



FIGURE 3.1 – Diagramme des cas d'utilisation : interactions entre Administrateur, Superviseur et Étudiant

Chapitre 4

Parcours utilisateurs critiques

4.1 Parcours étudiant

Le parcours étudiant débute après la création de son compte par un superviseur. L'étudiant reçoit un courriel d'activation contenant un lien de réinitialisation de mot de passe. Après définition de son mot de passe, il accède à son espace personnel de la plateforme.

Une fois authentifié, l'étudiant consulte le stage auquel il est rattaché, les objectifs définis par le superviseur, et l'ensemble des tâches qui lui ont été assignées. Il met régulièrement à jour le statut de chaque tâche pour refléter sa progression réelle, et consulte les remarques pédagogiques et conseils fournis par le superviseur.

Le parcours s'achève par la rédaction et la soumission du rapport final. L'étudiant peut suivre l'état de son rapport au sein de la plateforme (en attente de revue, corrections demandées, approuvé, etc.). Toutes les actions de l'étudiant sont historisées afin de permettre une auditabilité complète et un suivi pédagogique documenté.

4.2 Parcours superviseur

Le parcours superviseur s'amorce dès son inscription et la création d'un stage. Le superviseur s'inscrit librement via l'authentification standard de la plateforme. Il crée ensuite un stage en fournissant la description, les dates, les objectifs pédagogiques et les références de la structure d'accueil.

Une fois le stage enregistré, le superviseur crée les comptes étudiants associés. Chaque création de compte déclenche l'envoi automatique d'un courriel d'activation permettant à l'étudiant de se connecter. Le superviseur définit ensuite le cadre opérationnel : il décompose la mission en tâches planifiées, fixe les échéances, et assigne chaque tâche à l'étudiant.

Tout au long du stage, le superviseur suit l'avancement des tâches, fournit des remarques régulières pour orienter la progression et soutenir l'apprentissage, identifie les écarts éventuels et propose des actions correctives. En fin de cycle, il traite le rapport soumis : il examine le contenu, valide ou demande des révisions en s'appuyant sur des critères explicites et pédagogiques. Sa décision finale garantit une clôture formelle et traçable du stage.

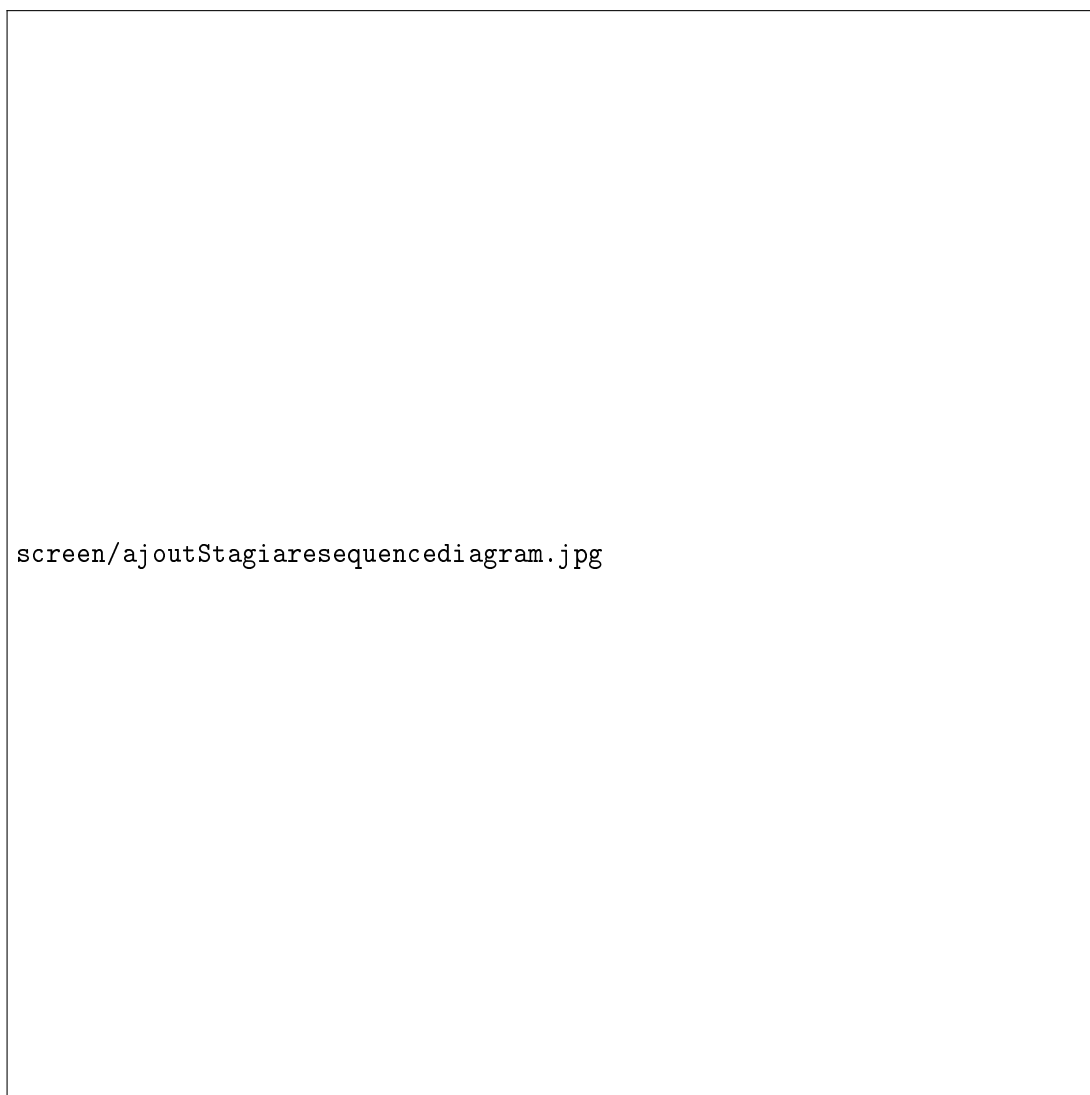


FIGURE 4.1 – Figure 3.1 : : Diagramme de séquence : création du compte étudiant et suivi de stage



FIGURE 4.2 – Figure 3.2 : : Diagramme d'activités : avancement et suivi des tâches entre étudiant et superviseur

Chapitre 5

Architecture technique

5.1 Architecture globale

StageFlow adopte une architecture client-serveur en trois couches logiques : présentation, services applicatifs et persistance des données.

La couche présentation est assurée par le frontend React/Vite, qui fournit des interfaces réactives, modulaires et adaptées à chaque rôle utilisateur. La couche services est implémentée avec Node.js/Express et expose une API REST structurée par domaines fonctionnels (authentification, stages, tâches, rapports, administration). La couche données repose sur PostgreSQL, garantissant l'intégrité relationnelle et la cohérence transactionnelle des opérations critiques.

Cette organisation favorise une séparation claire des responsabilités, améliore la maintenabilité du code et facilite l'évolutivité de la plateforme. Les contrôleurs orchestrent la logique métier, les middlewares assurent les préoccupations transversales (authentification, validation, gestion d'erreurs, logging), et la base de données matérialise les règles d'intégrité via contraintes de clés étrangères, index adaptés et historisation des actions critiques.

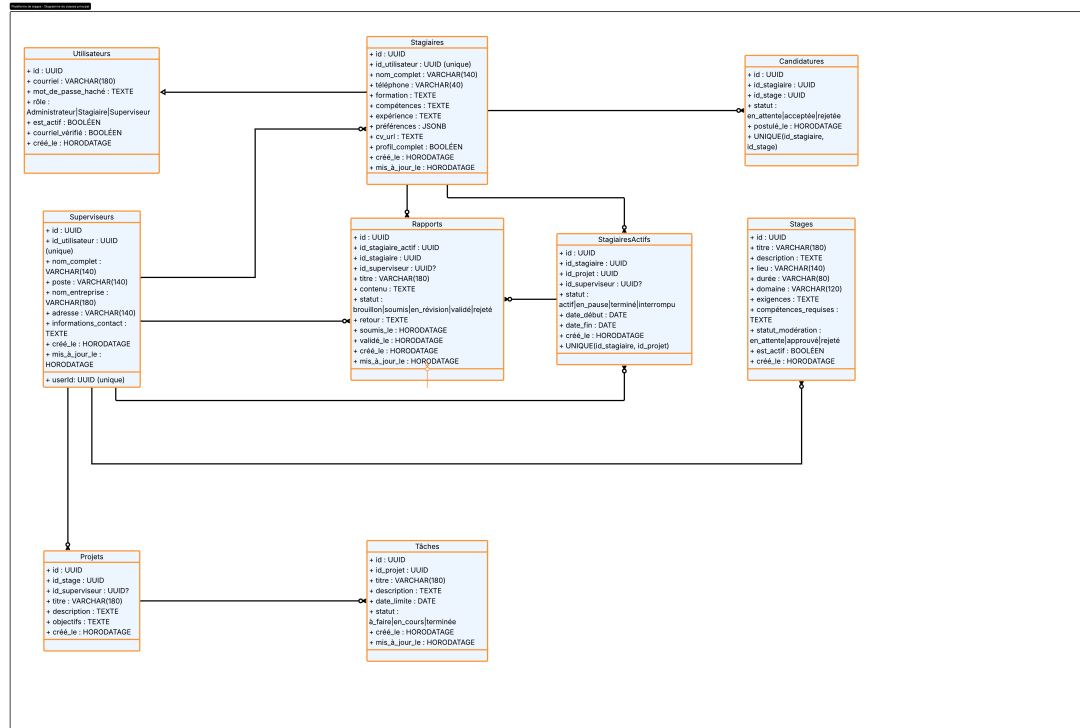


FIGURE 5.1 – Figure 4.1 : : Diagramme de classes UML : entités principales et relations

5.2 Stack technologique

Le choix de la stack technologique a été guidé par des critères de performance, de productivité, de maintenabilité et de sécurité dans un contexte full-stack moderne.

Frontend : React et Vite permettent de construire une interface réactive, modulaire et rapide au chargement. Axios est utilisé pour la communication HTTP avec l'API backend, avec une gestion centralisée des en-têtes d'authentification JWT et une gestion cohérente des erreurs réseau.

Backend : Node.js et Express offrent un socle léger et efficace pour implémenter une API REST. Joi est utilisé pour la validation stricte et déclarative des données entrantes, bloquant les incohérences dès la couche d'entrée. JWT soutient l'authentification stateless et sécurisée, tandis que le modèle RBAC (Role-Based Access Control) encadre les autorisations selon le rôle de l'utilisateur connecté.

Base de données : PostgreSQL assure une persistance robuste et fiable des données métier. Son modèle relationnel facilite la traçabilité des workflows critiques liés aux stages, aux tâches, aux remarques et aux rapports. Les index ciblés et les contraintes d'intégrité optimisent les performances et garantissent la cohérence.

5.3 Sécurité et validation

La validation des données est implémentée via Joi au niveau des routes, bloquant les requêtes invalides avant traitement. L'authentification JWT protège l'accès aux ressources critiques de l'API, tandis que RBAC garantit qu'un utilisateur n'accède qu'aux opérations compatibles avec son rôle métier. Les mots de passe sont hachés via bcrypt. Les logs des actions sensibles sont enregistrés pour l'auditabilité et l'investigation en cas d'incident.

Chapitre 6

Interfaces utilisateur et captures d'écran

Le Chapitre 7 « Interfaces utilisateur et illustrations » devient un chapitre entièrement dédié aux captures d'écran de l'application StageFlow. Ce chapitre ne contiendra que les interfaces les plus importantes et représentatives du fonctionnement réel de la plateforme, afin d'illustrer concrètement les fonctionnalités principales implémentées.

Sélectionner uniquement les captures pertinentes : authentification, tableau de bord administrateur, gestion des stagiaires, création de stage, gestion des tâches, suivi de progression, espace étudiant, gestion des rapports et workflow d'encadrement. Éviter les captures répétitives ou secondaires afin de conserver une présentation professionnelle, lisible et cohérente avec un rapport de mémoire.

Chaque capture doit être accompagnée :

- d'un titre académique clair ;
- d'une courte description fonctionnelle ;
- d'une explication du rôle de l'interface dans le workflow global de StageFlow.

Les images seront insérées ci-dessous lorsque vous les fournirez. Pour l'instant chaque section contient une zone réservée (placeholder) indiquant l'image attendue et laissant la place pour l'insertions ultérieure.

6.1 7.1 Authentification — Écran de connexion

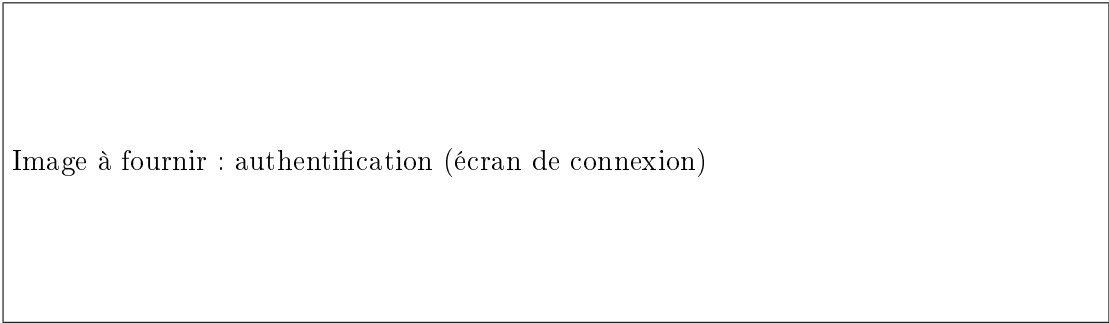


Image à fournir : authentification (écran de connexion)

FIGURE 6.1 – Authentification — Écran de connexion (image fournie ultérieurement)

Description fonctionnelle Écran permettant à un utilisateur (Administrateur, Superviseur, Étudiant) de se connecter via identifiants protégés. Gère les erreurs d'authentification et les redirections selon le rôle.

Rôle dans le workflow Point d'entrée sécurisé de la plateforme ; contrôle l'accès aux ressources en appliquant le RBAC et initialise le contexte utilisateur pour les parcours suivants.

6.2 7.10 Sécurité, fiabilité et qualité

6.2.1 Mécanismes de sécurité

La sécurité de StageFlow est traitée comme une exigence transversale, intégrée à tous les niveaux de l'application. L'authentification par JWT protège les accès aux ressources API, tandis que l'autorisation RBAC garantit qu'un utilisateur n'accède qu'aux opérations compatibles avec son rôle métier. Cette combinaison réduit significativement le risque d'exposition involontaire de données sensibles.

En complément, la validation côté serveur (Joi) bloque les entrées invalides ou malveillantes avant leur traitement. Les mécanismes de journalisation (logging) enregistrent les actions sensibles, facilitant l'investigation en cas d'incident et renforçant l'auditabilité du système. La gestion centralisée des erreurs limite les fuites d'information technique dans les réponses renvoyées au client, prévenant l'exposition involontaire de détails d'implémentation.

6.2.2 Qualité et robustesse

La qualité de la plateforme repose sur la cohérence des workflows métier, la stabilité des composants techniques et la vérification continue du comportement applicatif. Les tests fonctionnels par rôle permettent de valider les scénarios critiques de bout en bout : création et gestion des stages, création des comptes étudiants, assignation et suivi des tâches, suivi de l'avancement et validation des rapports.

La robustesse est renforcée par une structuration claire du code (routes, contrôleurs, middlewares, validations), une gestion explicite et documentée des erreurs, et une modélisation relationnelle cohérente dans PostgreSQL. Des revues régulières des dépendances et des configurations de sécurité contribuent à limiter la dette technique et à maintenir un niveau de fiabilité compatible avec un usage réel en environnement académique et professionnel.

6.3 7.2 Tableau de bord administrateur



FIGURE 6.2 – Tableau de bord Administrateur (image fournie ultérieurement)

Description fonctionnelle Vue synthétique des indicateurs système : nombre d'utilisateurs, stages actifs, alertes et accès rapides aux fonctions d'administration.

Rôle dans le workflow Permet à l'administrateur de superviser l'état global du système, gérer les comptes et intervenir en cas d'incident ou d'anomalie.

6.4 7.3 Gestion des stagiaires



FIGURE 6.3 – Gestion des stagiaires — liste et fiche (image fournie ultérieurement)

Description fonctionnelle Interface listant les stagiaires, permettant la création/modification des fiches et l’affectation aux stages.

Rôle dans le workflow Outil central pour le superviseur : associer un étudiant à un stage, suivre ses informations et historique administratif.

6.5 7.4 Création de stage

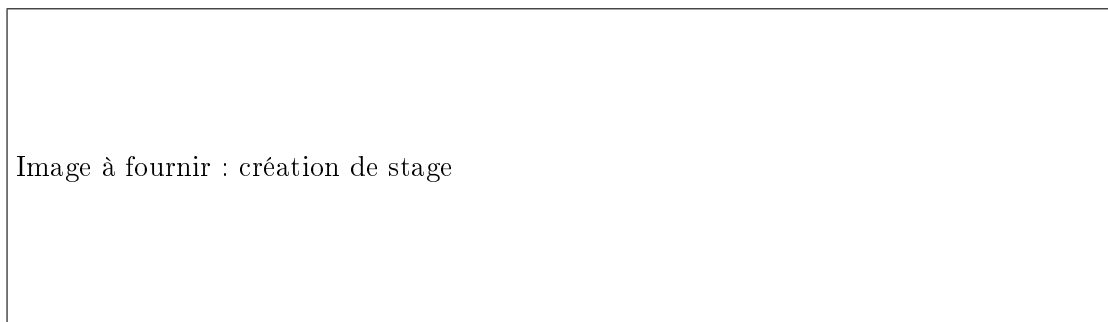


FIGURE 6.4 – Création de stage — formulaire de saisie (image fournie ultérieurement)

Description fonctionnelle Formulaire de création d’un stage : titre, description, dates, objectifs pédagogiques et superviseur référent.

Rôle dans le workflow Permet d’enregistrer l’entité stage qui servira de conteneur aux tâches, au suivi et à la production du rapport final.

6.6 7.5 Gestion des tâches

Description fonctionnelle Interface permettant la création, l’assignation, la mise à jour des statuts et la remise de livrables pour chaque tâche.

Rôle dans le workflow Outil opérationnel pour découper la mission en tâches, suivre l’avancement et consigner les retours de l’encadrement.

Image à fournir : gestion des tâches

FIGURE 6.5 – Gestion des tâches — création et suivi (image fournie ultérieurement)

6.7 7.6 Suivi de progression

Image à fournir : suivi de progression

FIGURE 6.6 – Suivi de progression — indicateurs et timeline (image fournie ultérieurement)

Description fonctionnelle Vue consolidée des indicateurs de progression : tâches achevées, jalons atteints, et tendances temporelles.

Rôle dans le workflow Permet d'évaluer l'avancement global du stagiaire et d'anticiper les actions de remédiation si nécessaire.

6.8 7.7 Espace étudiant

Image à fournir : espace étudiant

FIGURE 6.7 – Espace étudiant — tableau personnel et soumission de livrables (image fournie ultérieurement)

Description fonctionnelle Tableau de bord personnel de l'étudiant : tâches assignées, échéances, messages et accès au formulaire de rédaction/soumission du rapport.

Rôle dans le workflow Interface principale de l'étudiant pour exécuter le travail, soumettre des livrables et consulter les retours du superviseur.

6.9 7.8 Gestion des rapports

Image à fournir : gestion des rapports

FIGURE 6.8 – Gestion des rapports — soumission et workflow de revue (image fournie ultérieurement)

Description fonctionnelle Interface de soumission et de revue du rapport de stage, avec historique des versions et commentaires du superviseur.

Rôle dans le workflow Conclut le cycle pédagogique : réception, revue et validation formelle du rapport.

6.10 7.9 Workflow d'encadrement (vue synthétique)

Image à fournir : workflow d'encadrement — synthèse

FIGURE 6.9 – Workflow d'encadrement — synthèse des interactions (image fournie ultérieurement)

Description fonctionnelle Vue synthétique montrant les interactions essentielles entre superviseur et étudiant autour des tâches, des retours et de la validation des livrables.

Rôle dans le workflow Fournit une vision d'ensemble utile pour les jurys et la documentation académique.

Chapitre 7

Outils et technologies utilisés

Ce chapitre présente les principaux outils, frameworks et technologies employés dans le développement de StageFlow. Pour chaque technologie, nous décrivons son rôle dans le projet, ses avantages dans l'architecture retenue et son intégration dans le workflow de développement. Chaque section inclut une zone réservée pour une illustration ou capture d'écran représentative.

7.1 8.1 Git et GitHub

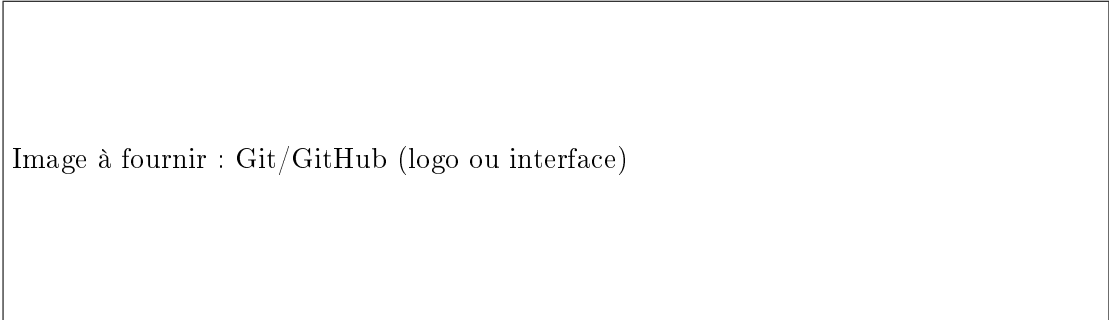


Image à fournir : Git/GitHub (logo ou interface)

FIGURE 7.1 – Git et GitHub — contrôle de version distribué (image fournie ultérieurement)

Description technique Git est un système de contrôle de version distribué qui permet de suivre les modifications du code source au fil du temps. GitHub est une plateforme collaborative en ligne basée sur Git, offrant un dépôt centralisé, des outils de collaboration et de l'intégration continue.

Rôle et avantages dans StageFlow Git soutient le processus de développement itératif : chaque modification de code est enregistrée avec un message explicite, permettant de tracer l'historique complet du projet. GitHub facilite la collaboration entre développeurs, offre un mécanisme de revue de code via les pull requests, et constitue une archive permanente et distribuée du codebase. La possibilité de créer des branches isolées permet de développer des fonctionnalités en parallèle sans affecter la branche principale (**master**), assurant une stabilité et une sérénité du déploiement.

Intégration dans le workflow Lors du développement, chaque ensemble cohérent de modifications est committée avec un message clair (ex. : **fix: authentication JWT**). Les commits sont poussés régulièrement vers le dépôt distant (GitHub), constituant une sauvegarde et un point d'ancrage pour la collaboration. GitHub Actions peut être configurée pour déclencher des tests automatiques ou des déploiements à chaque push, renforçant l'assurance qualité.

7.2 8.2 Node.js

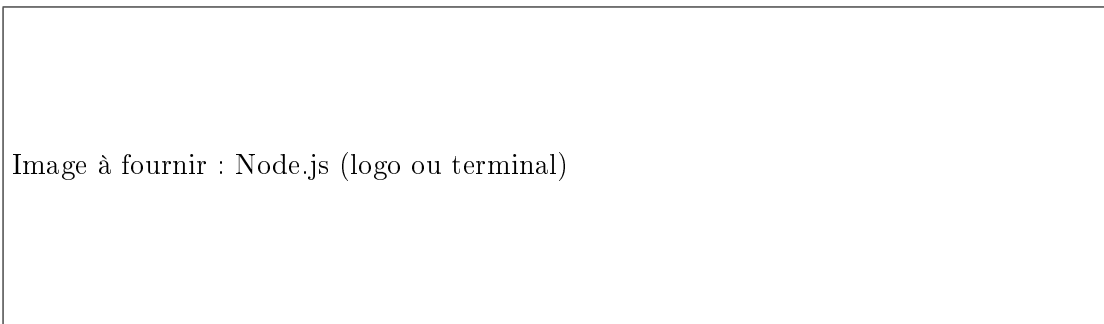


FIGURE 7.2 – Node.js — runtime JavaScript côté serveur (image fournie ultérieurement)

Description technique Node.js est un runtime JavaScript asynchrone et orienté événements qui permet d'exécuter du JavaScript côté serveur, en dehors du navigateur. Construit sur le moteur V8 de Chrome, Node.js offre une performance élevée et une gestion efficace des I/O asynchrones.

Rôle et avantages dans StageFlow Le choix de Node.js pour le backend permet une cohérence du langage (JavaScript/TypeScript sur le frontend et backend), réduisant la friction cognitive et facilitant le partage de code utilitaire. L'architecture orientée événements de Node.js est particulièrement adaptée aux applications web modernes avec des milliers de connexions concurrentes. La gestion native des Promises et de `async/await` rend le code asynchrone lisible et maintenable. L'écosystème npm offre des millions de paquets réutilisables, accélérant le développement.

Intégration dans le workflow Le serveur backend de StageFlow est lancé via la commande `npm run dev`, qui démarre un serveur Node.js à l'écoute sur le port 3000. Durant le développement, les modifications du code sont détectées automatiquement (via nodemon), ce qui redémarre le serveur sans interruption utilisateur. La gestion des modules via npm (définie dans `package.json`) centralise toutes les dépendances.

7.3 8.3 Express.js

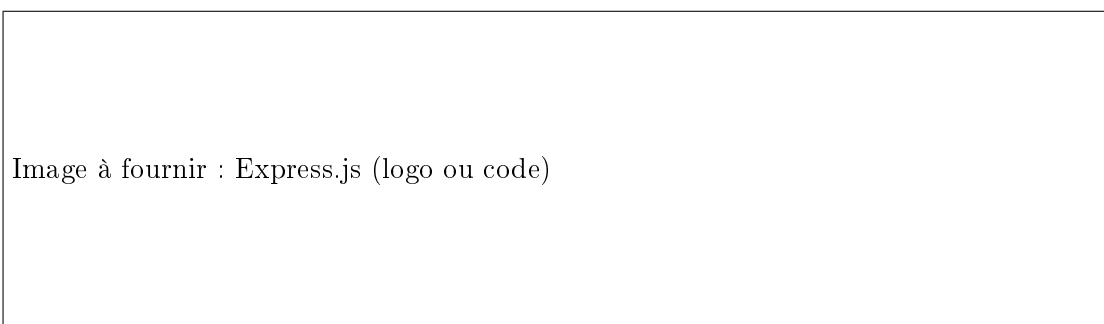


FIGURE 7.3 – Express.js — framework web minimaliste (image fournie ultérieurement)

Description technique Express.js est un framework web léger et flexible pour Node.js, conçu pour construire des applications web et des API REST. Il fournit un système de routage, un middleware réutilisable et des outils essentiels pour la gestion des requêtes HTTP.

Rôle et avantages dans StageFlow Express est le cœur de l'architecture backend de StageFlow. Il offre un système de routage expressif et flexible permettant de définir les endpoints API (ex. : `POST /api/auth/login`) avec des handlers clairs. Le système middleware permet de chaîner des transformations (authentification, validation, logging) de manière déclarative et réutilisable. Express est très mature, bénéficie d'une large communauté, et s'intègre bien avec les outils d'écosystème Node.js.

Intégration dans le workflow Les routes de StageFlow sont organisées par domaine (auth, students, supervisors, internships, tasks, reports) dans `backend/src/routes/`. Chaque route s'appuie sur des contrôleurs pour la logique métier et des middlewares pour la validation et l'authentification. Cette structuration soutient la modularité et la testabilité du code.

7.4 8.4 PostgreSQL



Image à fournir : PostgreSQL (logo ou dashboard)

FIGURE 7.4 – PostgreSQL — système de gestion de base de données relationnelle (image fournie ultérieurement)

Description technique PostgreSQL est un système de gestion de base de données relationnelle (SGBDR) open-source, réputé pour sa robustesse, sa conformité aux standards SQL et ses capacités avancées (transactions ACID, triggers, jsonb, extensions, etc.).

Rôle et avantages dans StageFlow PostgreSQL assure la persistance fiable et cohérente des données métier de StageFlow. Son modèle relationnel permet de modéliser précisément les entités (utilisateurs, étudiants, superviseurs, stages, tâches, rapports) et leurs relations. Les contraintes d'intégrité référentielle (clés étrangères) garantissent la cohérence des données. Les transactions ACID assurent que les opérations critiques (approbation de rapport, assignation de tâche) sont atomiques et cohérentes. Les index ciblés optimisent les requêtes fréquentes.

Intégration dans le workflow Le code backend se connecte à PostgreSQL via le driver `pg` (node-postgres), configuré dans `backend/src/config/db.js`. Les migrations SQL dans `backend/migrations/` gèrent l'évolution du schéma de façon versionnée. Des scripts de seed (`backend/scripts/seed*.js`) peuplant la base avec des données de démonstration facilitent les tests et l'onboarding.

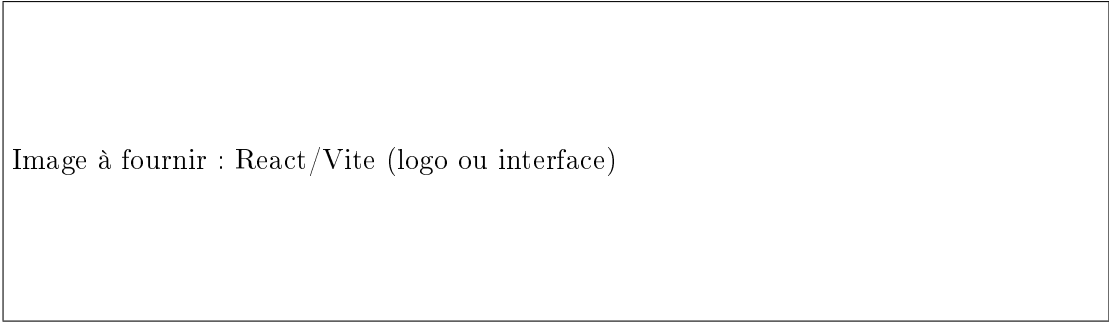


Image à fournir : React/Vite (logo ou interface)

FIGURE 7.5 – React et Vite — framework frontend et bundler (image fournie ultérieurement)

7.5 8.5 React et Vite

Description technique React est une bibliothèque JavaScript pour construire des interfaces utilisateur réactives et composables. Vite est un bundler de module ES moderne offrant un développement ultra-rapide avec remplacement de module à chaud (HMR).

Rôle et avantages dans StageFlow React permet de découper l'interface utilisateur en composants réutilisables et testables (FormField, StatusBadge, InfoCard, Layout, etc.), favorisant la maintenabilité et la cohérence du design. La programmation déclarative de React rend le code plus lisible et facile à déboguer. Vite offre des temps de compilation extrêmement rapides et un rechargement instantané du code lors du développement, améliorant l'expérience développeur.

Intégration dans le workflow L'application frontend de StageFlow est développée en React, avec des pages par domaine (LoginPage, DashboardPage, TasksPage, ReportsPage, etc.). Vite configure le bundling et le serveur de développement via `vite.config.js`. Lors du développement, `npm run dev` démarre un serveur Vite à l'écoute sur le port 5173, avec HMR activée. Pour la production, `npm run build` génère des assets statiques optimisés.

7.6 8.6 Docker

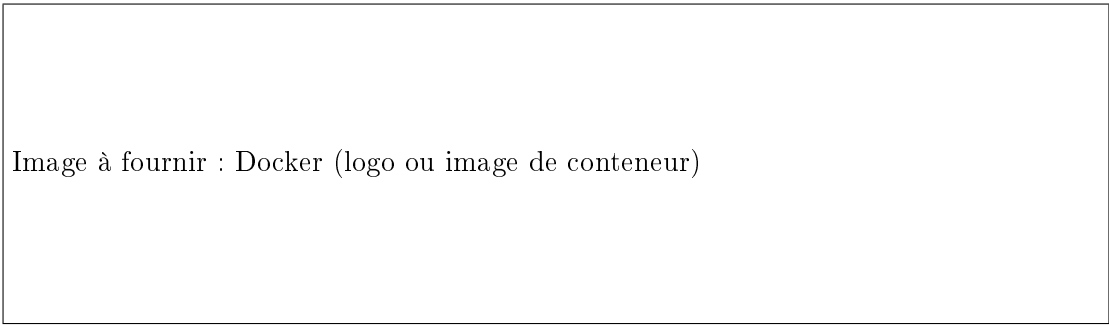


Image à fournir : Docker (logo ou image de conteneur)

FIGURE 7.6 – Docker — conteneurisation et orchestration (image fournie ultérieurement)

Description technique Docker est une plateforme de conteneurisation qui empaquète une application et ses dépendances (runtime, bibliothèques, configurations) dans une unité exécutable isolée appelée conteneur.

Rôle et avantages dans StageFlow Docker assure la reproductibilité de l'environnement entre développement, test et production. Chaque service (backend, frontend, base de données) est défini dans un Dockerfile et can be built en une image réutilisable. Docker Compose orchestre plusieurs conteneurs, facilitant le déploiement local de toute la pile (backend + frontend + PostgreSQL) avec une simple commande : `docker compose up -d -build`.

Intégration dans le workflow Le fichier `docker-compose.yml` à la racine du projet définit les trois services : *backend* (Node.js/Express), *frontend* (nginx + assets React), et *db* (PostgreSQL). Lors du déploiement en production, les images Docker sont construites une fois et déployées sur des infrastructures cloud (AWS ECS, Azure Container Instances, etc.), garantissant une cohérence stricte.

7.7 8.7 JWT (JSON Web Token)

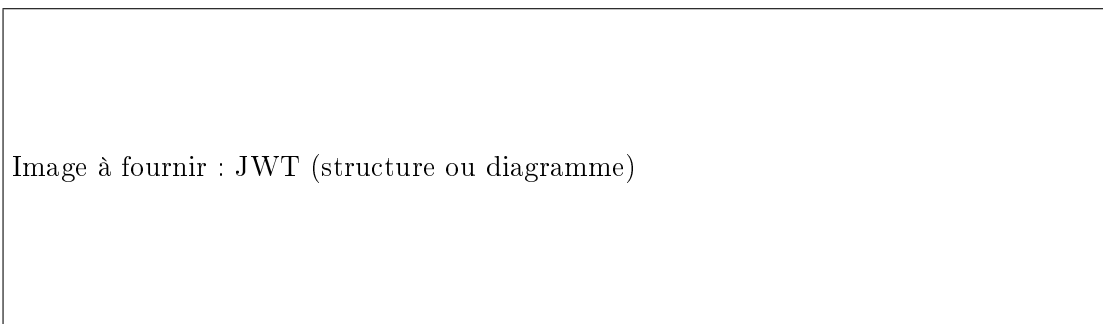


FIGURE 7.7 – JWT — authentification par jeton stateless (image fournie ultérieurement)

Description technique JWT est un standard (RFC 7519) pour créer des jetons autonomes contenant des assertions (claims) signées cryptographiquement. Un JWT se compose de trois parties : header (type et algorithme), payload (données/claims), et signature.

Rôle et avantages dans StageFlow JWT soutient l'authentification stateless de StageFlow : une fois qu'un utilisateur se connecte avec succès (identifiant + mot de passe), le serveur génère un JWT contenant l'ID utilisateur, le rôle, et d'autres métadonnées pertinentes. Ce jeton est stocké côté client (localStorage ou cookie) et envoyé avec chaque requête API. Le serveur vérifie la signature du jeton pour confirmer son authenticité sans consulter la base de données. Cette approche évite le stockage d'état sur le serveur (sessions), permettant une scalabilité horizontale sans synchronisation d'état.

Intégration dans le workflow Le middleware d'authentification de StageFlow (défini dans `backend/middlewares/authMiddleware.js`) vérifie le JWT avant chaque accès à une ressource protégée. Si le jeton est valide, la requête continue avec l'identité de l'utilisateur attachée au contexte ; sinon, une réponse 401 Unauthorized est renvoyée. Des jetons de courte durée (ex. 15 minutes) associés à des jetons de renouvellement (refresh tokens) de longue durée offrent un équilibre entre sécurité et expérience utilisateur.

7.8 8.8 Joi

Description technique Joi est une bibliothèque JavaScript pour la validation de données et la gestion de schémas, offrant une API déclarative et expressive pour définir et valider la structure des objets.

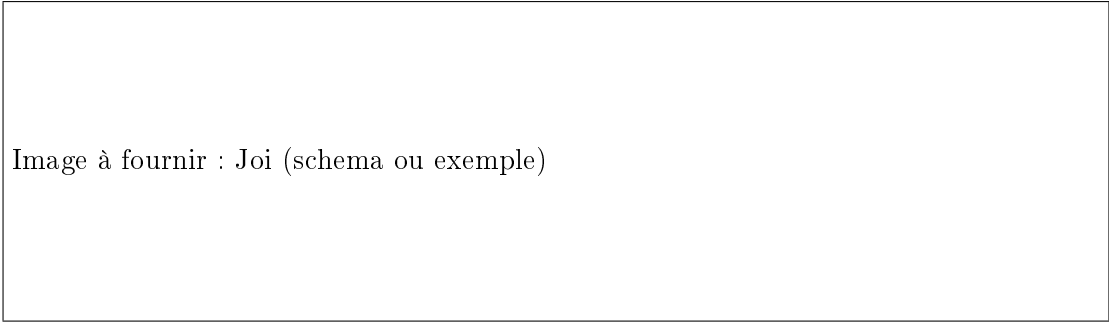


Image à fournir : Joi (schema ou exemple)

FIGURE 7.8 – Joi — validation de schémas et données (image fournie ultérieurement)

Rôle et avantages dans StageFlow Joi assure que toutes les données entrantes (requêtes HTTP) respectent les règles métier et les contraintes de type avant d’être traitées par la logique applicative. Plutôt que de parser et valider manuellement chaque champ, Joi permet de définir des schémas déclaratifs (ex. : l’email doit être une chaîne valide, le mot de passe au moins 8 caractères). Si la validation échoue, une réponse d’erreur détaillée est renvoyée, prévenant les traitements d’entrées invalides et améliorant la sécurité.

Intégration dans le workflow Chaque route API de StageFlow est protégée par une validation Joi. Par exemple, la route POST `/api/auth/login` utilise un schéma Joi pour vérifier que l’email et le mot de passe sont présents et valides. Les schémas Joi sont centralisés dans `backend/src/validations/` (`authValidation.js`, `internValidation.js`, etc.) et appliqués via le middleware `validate` définissable au niveau des routes. Cette approche décentralise la logique de validation de la logique métier, favorisant la clarté.